CPTG Reaction Exchange Protocol

CPTG-RX/1.0

Detailed Framework Specification for Scientific Translation, Live Comparison Coordinates, and Institutional Integration

AUTHORITY BOUNDARY **This revision treats the universal CPTG light-element reaction theory as closed and places the exchange protocol under implementation authority. The scientific formula payload remains controlled and is referenced only through immutable identifiers, manifests, hashes, authorized domains, and conformance records. The protocol does not disclose formulas, certify any reactor controller, or authorize direct actuator commands.**

Prepared for the CPTG Nuclear Reaction Theory program

Revision r02 | 13 July 2026 | Locked-theory implementation authority

Core principle

The closed CPTG theory remains immutable and authoritative. Transports and facility adapters are replaceable. External models communicate through a published CPTG state/result contract without receiving or depending on the protected internal formula payload.

Document Control

Field	Value
Document title	CPTG Reaction Exchange Protocol (CPTG-RX/1.0) - Detailed Framework Specification
Revision	r02
Date	2026-07-13
Status	Locked-theory integration framework; implementation and conformance development authorized
Protocol scope	Scientific state/result exchange, live comparison coordinates, batch trajectories, evidence and replay
Out of scope	Safety certification, autonomous actuator authority, machine protection, or disclosure of protected formula payloads
Primary authority	Published protocol specification, schemas, formula manifests, conformance vectors, and immutable hashes
Reference implementation	Python/JAX scientific authority implementation, followed by a deterministic C-compatible runtime kernel
Canonical evidence format	HDF5 companion profile (.cptg.h5)
Proposed binary data encoding	FlatBuffers
Proposed transports	Shared memory or IPC for same-host use; ZeroMQ profile for facility/network integration; HTTPS/OpenAPI for control plane

Normative Status

This framework uses the key words MUST, MUST NOT, REQUIRED, SHALL, SHALL NOT, SHOULD, SHOULD NOT, RECOMMENDED, MAY, and OPTIONAL to distinguish mandatory interoperability requirements from implementation choices. CPTG-RX/1.0 follows the conventional RFC interpretation of these terms. The scientific theory is treated as locked for this revision. Requirements tied to a specific scientific output remain MANIFEST-DEPENDENT and must be satisfied by the corresponding immutable authority package.

Contents

1 Executive Summary

CPTG-RX is the institution-facing application protocol for the closed CPTG universal light-element reaction theory. It accepts a physical state vector and returns authorized native quantities, external comparison coordinates, sensitivities, validity information, uncertainty, timing, and provenance with minimal overhead. The authority covers the hydrogen, deuterium, mass-three, and helium-4 vertex-bridge-closure-saturation architecture, its ordered transport currents, and its charge-constrained polarization response, while keeping the protected formula payload outside this public specification.

The protocol is deliberately independent of any single transport, control framework, programming language, or reactor facility. Python is the reference scientific implementation, not the protocol itself. A laboratory may build its own EPICS, MARTe2, MHD, PCS, C++, Rust, Fortran, Julia, MATLAB, or custom adapter, provided that the adapter obeys the CPTG-RX schemas, formula manifests, sequence rules, unit conventions, validity gates, and conformance vectors.

The protocol is divided into three planes: a control plane for discovery and configuration, a real-time data plane for repeated StateFrame/ResultFrame exchange, and an evidence/replay plane for immutable scientific records. This separation preserves CPTG speed in the hot path without sacrificing traceability or interoperability.

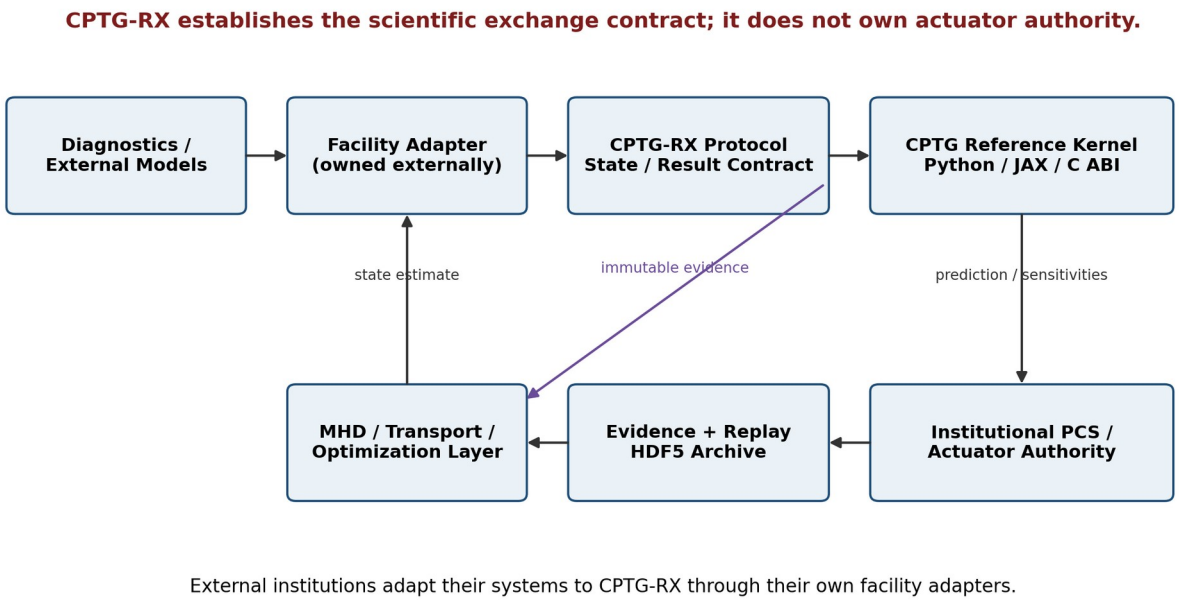

Figure 1. CPTG-RX owns the scientific exchange contract. External institutions retain ownership of facility integration, MHD/transport coupling, and actuator authority.

IMPLEMENTATION AUTHORITY The scientific theory is treated as closed for protocol development. Production release still requires immutable formula and constants packages, authorized domains, a precision contract, published conformance vectors, security qualification, implementation-specific replay, and independent client compatibility. These are release gates for the protocol implementation, not invitations to alter the theory.

2 Mission, Scope, and Boundaries

2.1 Mission

CPTG-RX shall provide a stable, high-speed translation boundary between CPTG reaction theory and external scientific or engineering systems. It is designed for use cases ranging from offline comparison studies to live plasma-state evaluation, while preserving exact formula identity and a complete evidence chain.

2.2 In scope

- Formula discovery, capability negotiation, and formula-manifest retrieval.
- Scalar, vector, batch, and trajectory evaluation of locked CPTG formulas.
- Native CPTG outputs and explicitly separated external comparison coordinates.
- First and second derivatives where the formula authority permits them.
- Domain validation, uncertainty propagation, status reporting, and deterministic failure.
- Institution-facing binary and HTTP profiles.
- Canonical evidence, replay, and audit files.
- Conformance testing for independent implementations.
- Adapters to facility data systems, MHD solvers, optimization systems, and control frameworks.

2.3 Out of scope

- Direct ownership of plasma actuators, magnetic-coil commands, machine protection, or emergency shutdown.
- Certification that a facility control loop is safe, stable, or regulator-approved.
- Use of pending or exploratory formulas as if they were locked scientific authority.
- Silent substitution of external nuclear rates, fits, or calibration constants.
- Silent extrapolation beyond the authorized CPTG domain.
- Requiring outside institutions to use CPTG internal Python code or a single vendor transport.

SEPARATION OF AUTHORITY CPTG-RX may calculate source terms, yields, cross-sections, reactivities, and sensitivities. The facility controller decides whether and how those quantities influence actuators. This boundary is mandatory.

3 Design Principles

Principle	Requirement
Protocol over implementation	The published state/result contract is authoritative. Python, C, JAX, ZeroMQ, shared memory, and HDF5 are implementations or profiles, not the scientific standard.
Immutable formula identity	Every result is bound to one formula ID, semantic revision, exact expression, constants set, authorized domain, and SHA-256 digest.
Native before comparison	Native CPTG outputs are returned separately from external comparison coordinates. No comparison value may masquerade as a native prediction.
Fail closed	Invalid, stale, malformed, out-of-domain, non-finite, or hash-inconsistent inputs produce explicit failure. They never produce a plausible-looking substitute.
Minimal hot path	The real-time path preloads formulas and constants, allocates memory before repeated evaluation, avoids file access, and returns only requested outputs.
Transport independence	The same payload semantics apply over shared memory, IPC, ZeroMQ, or a remote gateway.
Reproducibility by construction	Every archived result can be replayed from the request, formula package, constants, software build, precision mode, and

Principle	Requirement
	environment metadata.
Schema evolution without ambiguity	Fields are added through versioned schemas and compatibility rules; identifiers are never silently repurposed.
Institutional autonomy	External organizations own their adapter and control architecture while conforming to CPTG-RX.
Performance must be measured	Speed claims require reproducible p50, p95, p99, maximum, jitter, throughput, allocation, and environment results.

4 System Architecture

CPTG-RX separates configuration, repeated calculation, and evidence capture because the fastest scientific kernel can be overwhelmed by serialization, logging, network negotiation, or repeated initialization. The protocol therefore defines three operational planes.

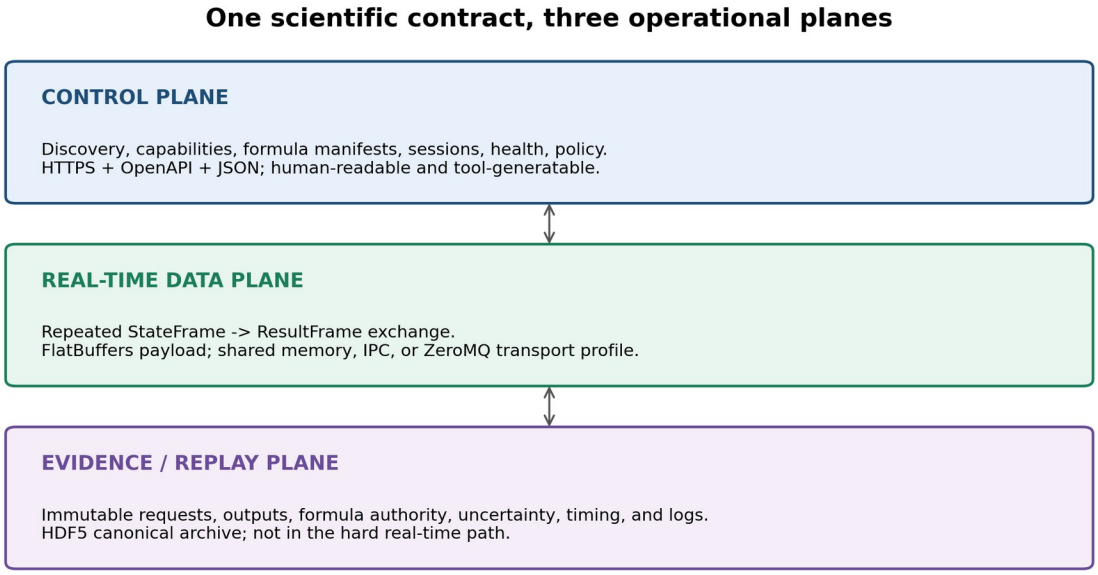

Figure 2. Control, real-time data, and evidence/replay planes share one scientific contract but use different operational mechanisms.

4.1 Control plane

The control plane is human-readable and tool-generatable. It is used for service discovery, protocol and schema negotiation, formula manifests, session configuration, health, policy, and administrative status. The recommended interface is HTTPS with OpenAPI and JSON. OpenAPI defines a language-agnostic description for HTTP APIs and allows consumers to understand service capabilities without inspecting source code [1].

4.2 Real-time data plane

The data plane carries repeated StateFrame and ResultFrame messages after a session has been established. It uses a compact binary payload, fixed identifiers, canonical units, sequence numbers, monotonic timestamps, and explicit deadlines. The recommended payload encoding is FlatBuffers because it supports in-place access and controlled schema evolution [2,3].

4.3 Evidence and replay plane

The evidence plane records complete requests, outputs, formula authority, validation, uncertainty, timing, and execution metadata in HDF5. HDF5 is suitable for heterogeneous scientific arrays and metadata, but it is not the hard real-time

network protocol [6]. Concurrent single-writer/multiple-reader use must respect HDF5 SWMR and filesystem requirements [7].

4.4 Adapter boundary

Facility adapters translate local telemetry and model structures into CPTG-RX StateFrame objects and translate ResultFrame objects into local model inputs. Adapters may be maintained by CPTG, a research institution, or a third party, but they must not alter native CPTG semantics or suppress validity status.

5 Scientific Contract and Formula Authority

5.1 Formula authority package

Every executable authority shall be accompanied by a signed or hash-verified formula authority package. The package is the scientific root of trust for CPTG-RX and must remain immutable after release. This public protocol does not reproduce the protected equations; deployments bind to controlled authority payloads through identifiers, manifests, hashes, authorized domains, and conformance records.

Manifest element	Requirement
formula_id	Globally unique stable identifier, e.g. CPTG.DYNAMIC_SOURCE_CURRENT.LIGHT4.
formula_version	Semantic version of the scientific formula, independent of software version.
formula_status	LOCKED, DEPRECATED, WITHDRAWN, or RESEARCH_ONLY.
exact_expression	Canonical UTF-8 symbolic expression or expression tree.
constants_set	Exact expressions, decimal values, units, and source authority for every constant.
authorized_domain	Closed or explicitly bounded domain for every required input and supported reaction channel.
output_definitions	Definitions, units, dimensions, sign conventions, and comparison-coordinate mappings.
precision_contract	Required input and internal precision; tolerance for conformance vectors.
validation_record	Named validation gates, held-out results, and known limitations.
formula_sha256	Digest over canonical manifest and exact formula payload.
constants_sha256	Digest over canonical constants payload.
test_vectors_sha256	Digest over the authoritative conformance vector set.

5.2 Locked-authority acceptance conditions

1. The formula has a defensible CPTG derivation rather than a post-hoc fit.
2. All constants are locked and represented exactly where possible.
3. The authorized domain is explicit and scientifically justified.
4. Native outputs and comparison-coordinate transforms are separately defined.
5. Reference vectors include interior, boundary, near-boundary, and invalid cases.
6. Numerical stability and precision requirements have been measured.
7. Held-out tests have passed under preregistered criteria.
8. Known limitations and prohibited extrapolations are documented.

5.3 Native and comparison coordinates

A ResultFrame shall contain two namespaces. native_outputs contains CPTG quantities directly produced by the locked theory. comparison_outputs contains translations into an external model, database, or coordinate convention. Every comparison output must name the reference model and the transform definition.

```
native_outputs:
  source_current
  curvature_response
  amplitude
  cross_section
  reactivity
  reaction_rate

comparison_outputs:
  reference_model_id
  coordinate_id
  reference_value
  cptg_value_in_reference_coordinate
  correction_factor
  residual
  transform_definition_id
```

5.4 Reaction and species registries

CPTG-RX shall use stable numeric identifiers for species, reaction channels, observables, and comparison coordinates. Human-readable names are provided by the manifest but are not repeatedly transmitted in the hot path. Registry entries are append-only; an identifier must never be reused for a different meaning.

6 Protocol Profiles and Session Lifecycle

Profile	Purpose	Recommended mechanism
CPTG-RX-HTTP	Discovery, manifest retrieval, small scalar requests, diagnostics	HTTPS + OpenAPI + JSON
CPTG-RX-RT	Low-latency repeated state/result exchange	FlatBuffers over shared memory, IPC, or ZeroMQ
CPTG-RX-BATCH	Large trajectories and parameter grids	Vectorized direct API, Arrow-compatible arrays, or framed binary batches
CPTG-RX-H5	Canonical evidence, replay, and institutional archive	HDF5 (.cptg.h5)
CPTG-RX-CABI	Deterministic in-process runtime integration	Stable C-compatible application binary interface

6.1 Session lifecycle

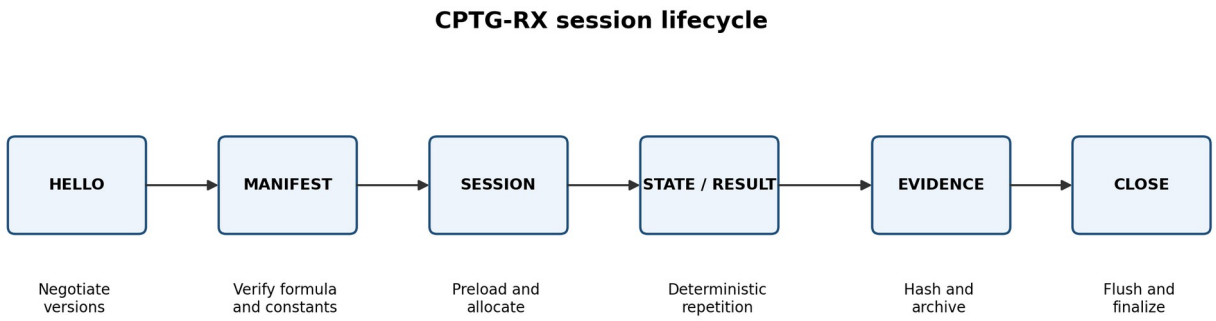

Figure 3. Session negotiation occurs once; formula verification, constant loading, and memory allocation are not repeated for every calculation.

Stage	Required actions
HELLO	Exchange protocol versions, schema IDs, encodings, precision support, maximum frame sizes, endianness, and client identity.
FORMULA_MANIFEST	Retrieve and verify formula, constants, domain, units, outputs, status, and hashes.
SESSION_CONFIG	Select formula, evaluation mode, precision, requested outputs, derivative level, transport, deadlines, and evidence policy.
PRELOAD	Load formula and constants; allocate buffers; warm or compile the kernel; run session self-test vectors.
STATE/RESULT	Exchange sequence-matched frames with no repeated discovery or file access.
EVIDENCE	Write selected frames and runtime metadata to a replay file or append-only evidence stream.
SESSION_CLOSE	Flush evidence, return final counters and hashes, invalidate session keys, and release preallocated resources.

6.2 Session immutability

A live session binds one formula manifest, constants set, schema version, precision mode, unit profile, and requested-output layout. Changing any of these requires a new session identifier. This prevents two scientific authorities from being mixed inside one time series.

7 Wire Framing and Binary Encoding

7.1 Fixed frame header

Every real-time message begins with a fixed-size header that can be validated before the payload is parsed. The header allows immediate rejection of incompatible versions, oversize payloads, stale sequences, and wrong formula hashes.

```
struct cptg_rx_header_v1 {
    char     magic[8];          /* "CPTGRX1" */
    uint16_t protocol_major;
    uint16_t protocol_minor;
    uint16_t message_type;
    uint16_t flags;
    uint32_t header_bytes;
    uint32_t payload_bytes;
    uint64_t session_id;
    uint64_t sequence_number;
    uint64_t monotonic_timestamp_ns;
    uint64_t deadline_timestamp_ns;
    uint32_t schema_id;
    uint32_t status_code;
    uint8_t  formula_sha256[32];
    uint8_t  payload_sha256[32];   /* optional in real-time mode */
};
```

7.2 Payload encoding

The normative v1 binary payload is proposed as FlatBuffers. FlatBuffers stores structured binary objects for in-place traversal, and its documented schema-evolution rules permit compatible additions when fields are appended and identifiers are managed correctly [2,3]. The protocol shall publish the .fbs schemas and generated-code compatibility matrix.

7.3 Endianness and alignment

The protocol header shall define one network representation. The FlatBuffers payload uses its specified little-endian representation [3]. Implementations on other architectures must convert as required. Numeric arrays shall be naturally aligned in direct-memory profiles.

7.4 Compression

Compression is forbidden for the smallest real-time frames unless benchmark evidence shows a net latency benefit. Compression may be negotiated for large batch or remote evidence transfers. The compression algorithm, version, level, and dictionary hash must be recorded.

7.5 Frame-size limits

Every session declares maximum header, scalar-frame, profile-frame, and batch-frame sizes. Oversize frames are rejected before allocation. Fixed upper bounds are required for deterministic runtime profiles.

8 Message Catalogue

Message	Direction	Function
HELLO	Client -> server	Negotiates protocol version, encoding, precision, limits, and identity.
CAPABILITIES	Server -> client	Lists supported formulas, profiles, schemas, precision modes, derivatives, and limits.
FORMULA_MANIFEST_REQUEST	Client -> server	Requests one immutable formula authority manifest.
FORMULA_MANIFEST	Server -> client	Returns formula authority, domain, outputs, and hashes.
SESSION_OPEN	Client -> server	Requests a formula-bound session.
SESSION_READY	Server -> client	Confirms preload, allocation, self-test, and session constraints.
STATE_FRAME	Client -> server	Supplies one scalar, profile, grid, or batch state.
RESULT_FRAME	Server -> client	Returns outputs, derivatives, uncertainty, validity, and timing.
ERROR_FRAME	Either direction	Reports deterministic structured failure.
HEARTBEAT	Either direction	Carries liveness, last sequence, counters, and health.
EVIDENCE_MARK	Client -> server	Requests evidence checkpoint or named event marker.
SESSION_CLOSE	Client -> server	Requests orderly shutdown.
SESSION_SUMMARY	Server -> client	Returns final counts, errors, timing quantiles, and evidence hashes.

8.1 Unknown messages

Unknown message types are rejected unless the session explicitly negotiated an extension namespace. Extensions may not redefine core messages or alter the meaning of required fields.

9 StateFrame Data Contract

StateFrame carries all physical and administrative inputs required to evaluate the selected formula. The manifest declares which fields are REQUIRED, OPTIONAL, or PROHIBITED for each formula and mode.

Field	Type	Status	Definition
sequence_number	uint64	required	Strictly increasing within a session.
measurement_timestamp_ns	uint64	required	Source measurement time in a declared clock domain.
validity_interval_ns	uint64	required	Maximum time for which the state remains usable.
reaction_channel_ids	uint32[]	required	Stable registry identifiers.
species_ids	uint32[]	required	Stable species registry identifiers.
number_density	float64[]	formula-dependent	SI m^-3 unless manifest specifies a canonical scaled coordinate.
ion_temperature	float64[]	formula-dependent	SI K; optional comparison coordinate T9 may be carried separately.
electron_temperature	float64	formula-dependent	SI K.
velocity_distribution_id	uint32[]	formula-dependent	Maxwellian, tabulated, beam, or declared custom distribution.
distribution_parameters	float64[]	formula-dependent	Parameters defined by distribution manifest.
spatial_grid	float64[]	optional	Canonical SI coordinates and grid metadata.
magnetic_field	float64[]	optional	SI tesla; profile or vector according to schema.
plasma_current	float64	optional	SI ampere.
q_profile	float64[]	optional	Dimensionless safety-factor profile, if facility adapter supplies it.
effective_charge	float64[]	optional	Dimensionless Z_eff profile.
external_heating_power	float64[]	optional	SI watt, separated by actuator/source ID.
fuel_injection_rate	float64[]	optional	SI particles/s or kg/s as manifest specifies.
requested_output_mask	bitset	required	Selects outputs and derivatives to minimize hot-path work.
client_context	bytes	optional	Opaque facility correlation token; not interpreted scientifically.

9.1 State normalization

The facility adapter is responsible for converting local units, coordinate conventions, species labels, and grid ordering into the canonical CPTG-RX representation. The server validates dimensions and bounds but must not guess missing conversions.

9.2 Sparse and profile states

The schema shall support scalar states, one-dimensional profiles, multidimensional grids, and sparse channel sets. All arrays carry explicit shape metadata. A shape mismatch is a hard failure.

9.3 No silent defaults

Any default allowed by a formula manifest must be returned in the ResultFrame under applied_defaults. If a required input is missing, evaluation does not proceed.

10 ResultFrame Data Contract

Field	Type	Requirement
sequence_number	uint64	Matches the originating StateFrame exactly.
input_timestamp_ns	uint64	Copied from the state.
start_timestamp_ns	uint64	Kernel start in the server monotonic clock.
completion_timestamp_ns	uint64	Kernel completion in the server monotonic clock.
execution_time_ns	uint64	Measured kernel or end-to-end time as labeled.
native_outputs	typed arrays	Formula-defined CPTG quantities.
comparison_outputs	typed records	Reference-model translations with transform IDs.
jacobian	typed matrix	Requested first derivatives with explicit variable ordering.
hessian	typed tensor	Requested second derivatives with explicit ordering.
uncertainty	typed records	Numerical, input, observational, and model-domain components kept separate.
domain_validity_mask	bitset	Per-row and per-output validity.
applied_defaults	typed records	Every default used during evaluation.
warnings	uint32[]	Non-fatal declared conditions.
status_code	uint32	One primary deterministic status.
formula_sha256	bytes[32]	Scientific authority digest.
result_hash	bytes[32]	Canonical result digest when enabled.
server_build_id	string/id	Reference implementation or runtime build identity.

10.1 Reaction outputs

Depending on the locked formula, native outputs may include coherent amplitude, source current, curvature response, cross-section, reactivity <sigma v>, reaction-rate density, species source terms, particle sinks, energy source density, gamma source, neutron source, helium-ash source, abundance coordinate, or yield. The manifest defines each field exactly.

10.2 Derivative metadata

Derivatives are returned with ordered variable and output identifiers. A raw matrix without ordering metadata is non-conformant. Analytic, automatic-differentiation, and finite-difference derivatives must be labeled because their uncertainty and performance differ.

10.3 Uncertainty decomposition

Uncertainty shall not be collapsed into one number unless a requested comparison coordinate explicitly requires it. At minimum, the protocol distinguishes numerical error, input measurement uncertainty, external comparison uncertainty, and theory-domain limitation.

11 Units, Precision, and Numerical Semantics

11.1 Canonical units

CPTG-RX shall use SI base and derived units as the canonical interchange representation, following the SI Brochure [8]. Formula-specific dimensionless coordinates such as T9 may be supported only when the manifest defines their exact conversion and the unit field remains explicit.

Quantity	Canonical representation
Time	second (s); monotonic packet timestamps in integer nanoseconds.
Length	metre (m).
Mass	kilogram (kg).
Temperature	kelvin (K).
Number density	m^-3.
Energy	joule (J) or electronvolt only with explicit unit code.
Power	watt (W).
Magnetic field	tesla (T).
Current	ampere (A).
Cross-section	m^2.
Reactivity	m^3 s^-1.
Reaction rate density	m^-3 s^-1.
Dimensionless quantity	Unit code DIMENSIONLESS; not an omitted unit.

11.2 Floating-point profiles

Profile	Input	Internal	Use
FP64-STRICT	binary64	binary64 or stronger	Default scientific and institutional profile.
FP32-RT	binary32	binary32 or mixed	Only if formula authority and error budget explicitly permit.
EXTENDED	binary64	extended or arbitrary precision	Reference generation and audit.
EXACT-SYMBOLIC	exact strings/rationals	symbolic + numerical	Formula and constants authority; not a general hot-path mode.

11.3 Non-finite values

NaN and infinity are rejected at the protocol boundary unless a formula manifest explicitly defines a diagnostic-only NaN input, which is not recommended. A non-finite intermediate causes a hard failure and is never serialized as a valid scientific result.

11.4 Rounding and reproducibility

Conformance vectors compare canonical unrounded binary values or declared high-precision decimals. Display rounding is not scientific authority. Implementations must disclose fused operations, compiler flags, and hardware modes when they affect last-bit reproducibility.

12 Timing, Determinism, and Throughput

12.1 Evaluation modes

Mode	Behavior
EXACT	Full validation, uncertainty, provenance, and all requested calculations. Suitable for reference and replay.
REALTIME	Preloaded formula, fixed buffers, no file access, bounded outputs, optional deferred evidence hashing.
BATCH	Vectorized trajectories or grids with row-level status and partial success.
DIAGNOSTIC	Verbose internals for development; prohibited in qualified real-time sessions.

12.2 Clock semantics

Human-readable UTC timestamps use the RFC 3339 Internet timestamp profile [10]. Real-time ordering and deadlines use a monotonic local clock encoded as integer nanoseconds. The two clocks are never substituted for one another.

12.3 Deadlines and stale data

Every StateFrame includes a validity interval or deadline. A server that begins or completes evaluation after the permitted deadline returns STALE_INPUT or DEADLINE_MISSED. A client must reject a ResultFrame whose sequence number does not match the intended control cycle.

12.4 Deterministic runtime rules

- No dynamic memory allocation after SESSION_READY in the strict real-time profile.
- No filesystem or network discovery in the repeated evaluation loop.
- No formula or constants change inside a live session.
- No logging that can block the kernel thread; evidence may use a bounded non-blocking queue.
- Fixed maximum array shapes and frame sizes.
- Declared thread affinity, compiler options, and numerical mode for benchmark claims.
- Explicit overload behavior: reject, drop according to policy, or route to a bounded queue; never expand without limit.

12.5 Latency budget

CPTG-RX does not impose one universal facility deadline. It reports separate kernel, serialization, queue, transport, and end-to-end latency. Each institution assigns deadlines appropriate to its control layer. The protocol preserves the formula speed advantage by making all overhead visible.

13 Validation, Fail-Closed Behavior, and Error Model

13.1 Validation sequence

9. Validate header magic, protocol version, sizes, schema ID, and session ID.
10. Validate sequence ordering, timestamp domain, validity interval, and deadline.
11. Validate formula hash against the session authority.
12. Parse payload and validate shapes, units, identifiers, and required fields.
13. Validate finite values and authorized-domain bounds.
14. Evaluate the formula using the selected precision and output mask.
15. Validate output finiteness, conservation conditions, and formula-specific invariants.
16. Construct ResultFrame, attach status and provenance, and optionally hash the canonical result.

13.2 Status classes

Code	Name	Meaning
0x0000	OK	Valid result.
0x0001	OK_WITH_WARNING	Result valid; warnings present.
0x1001	PROTOCOL_VERSION_UNSUPPORTED	No compatible protocol version.
0x1002	SCHEMA_UNSUPPORTED	Schema not recognized or not negotiated.
0x1003	MALFORMED_FRAME	Header or payload invalid.
0x1004	FRAME_TOO_LARGE	Declared or actual size exceeds session limit.
0x2001	FORMULA_NOT_FOUND	Requested formula unavailable.
0x2002	FORMULA_HASH_MISMATCH	Scientific authority does not match session.
0x2003	FORMULA_NOT_LOCKED	Formula status is not permitted for requested mode.
0x2004	CONSTANTS_HASH_MISMATCH	Constants package mismatch.
0x3001	MISSING_REQUIRED_INPUT	Required field absent.
0x3002	UNIT_ERROR	Unit missing, incompatible, or not canonicalizable.
0x3003	SHAPE_ERROR	Array shape or ordering invalid.
0x3004	NONFINITE_INPUT	NaN or infinity detected.
0x3005	OUT_OF_DOMAIN	Input outside authorized formula domain.
0x3006	STALE_INPUT	Validity interval expired.
0x4001	NUMERICAL_FAILURE	Evaluation failed or produced non-finite intermediate.
0x4002	INVARIANT_FAILURE	Conservation or formula-specific invariant failed.
0x4003	DEADLINE_MISSED	Result not completed before declared deadline.
0x4004	OVERLOAD_REJECTED	Bounded capacity unavailable.
0x5001	EVIDENCE_FAILURE	Evidence write failed; result policy determines validity.
0x5002	AUTHORIZATION_FAILURE	Client not authorized for formula or mode.

13.3 Warning discipline

Warnings are enumerated and machine-readable. Free-form text may accompany them for diagnostics but cannot replace codes. A warning must never downgrade an out-of-domain condition into a valid result.

13.4 Batch partial success

Batch results carry row-level status. Invalid rows do not contaminate valid rows, and the aggregate status reports whether the batch is complete, partial, or failed.

14 Security and Trust Boundaries

14.1 Threat model

CPTG-RX must protect formula identity, result integrity, client authorization, evidence authenticity, and availability. It does not assume that every network peer, adapter, or configuration source is trusted.

14.2 Remote transport

Remote control-plane and data-plane gateways should use TLS 1.3, which is designed to protect client/server communications against eavesdropping, tampering, and message forgery [11]. Mutual TLS is recommended for institutional deployments. Same-host shared-memory profiles use operating-system access control and a separate authenticated control channel.

14.3 Authentication and authorization

- Authenticate service and client identity.
- Authorize formulas, modes, maximum rates, batch sizes, and evidence access by role.
- Distinguish read-only discovery from evaluation and administrative operations.
- Record identity and authorization decision in evidence metadata.
- Rotate credentials without changing formula identity.
- Reject anonymous real-time sessions by default.

14.4 Replay and stale-frame protection

Session IDs, strictly increasing sequence numbers, deadlines, and optional message authentication prevent a captured old StateFrame or ResultFrame from being accepted as a current control-cycle result.

14.5 No actuator authority

MANDATORY TRUST BOUNDARY CPTG-RX output is scientific data. A facility adapter may present it to a transport solver, optimizer, or controller, but the protocol contains no command that directly actuates coils, beams, pumps, valves, or safety systems.

15 Provenance, Reproducibility, and Hashing

15.1 Hash standard

The baseline digest is SHA-256. NIST FIPS 180-4 specifies SHA-2 algorithms, including SHA-256, for generating message digests used to detect changes [9]. A future protocol revision may add stronger or signed manifests without removing the v1 digest fields.

15.2 Canonicalization

Hashes are meaningful only if canonical serialization is defined. CPTG-RX shall define canonical ordering, character encoding, floating-point representation, absent-field treatment, and array shape encoding for formula manifests, requests, and results.

Digest	Covers
formula_sha256	Canonical formula manifest, exact expression, domain, output definitions, and formula status.
constants_sha256	Exact and numerical constants payload, units, and constant IDs.
schema_sha256	Normative schema source and registry snapshot.
request_hash	Canonical StateFrame plus session scientific configuration.
result_hash	Canonical ResultFrame excluding the result_hash field itself.
evidence_file_hash	Finalized HDF5 file or a companion manifest of dataset/chunk hashes.
software_build_hash	Source tree and reproducible build identity where available.

15.3 Minimum provenance record

- Protocol and schema versions.

- Formula ID, version, status, and hashes.
- Constants, registry, and test-vector hashes.
- Reference or runtime software version and build hash.
- Operating system, architecture, compiler, numerical libraries, and device type.
- Precision mode and relevant compiler/runtime flags.
- Session configuration, request hash, result hash, and timing.
- Client and service identity, if policy permits archival.
- Validation status, warnings, and domain mask.

16 HDF5 Evidence and Replay Profile

16.1 Purpose

The .cptg.h5 profile is the canonical institutional evidence container. It is designed for complete replay, independent audit, shot or experiment archiving, benchmark comparison, and transfer of large multidimensional trajectories. HDF5 is a high-performance scientific data library and file format for heterogeneous data [6].

16.2 Normative group layout

```
/
  metadata/
    protocol_version
    schema_version
    session_id
    created_utc
    finalized_utc
    evidence_policy
  authority/
    formula_manifest_utf8
    formula_sha256
    constants_manifest_utf8
    constants_sha256
    schema_sha256
    registry_sha256
  inputs/
    state_frames
    shapes
    units
    source_identifiers
    request_hashes
  outputs/
    native/
    comparison/
    jacobians/
    hessians/
    status/
    result_hashes
  uncertainty/
    numerical/
    input/
    observational/
    domain/
  timing/
    measurement_timestamp_ns
    start_timestamp_ns
    completion_timestamp_ns
    execution_time_ns
    queue_time_ns
    transport_time_ns
  validation/
    domain_masks
    invariant_results
    warnings
    errors
  provenance/
    software_environment
    build_manifest
    client_identity
    server_identity
    execution_log
  replay/
    expected_outputs
    tolerances
    conformance_summary
```

16.3 Dataset rules

- Use explicit dtypes, shapes, units, and dimension labels.
- Chunk trajectory datasets according to dominant replay access patterns.
- Use compression only when the algorithm and parameters are recorded.
- Store exact symbolic constants and formulas as UTF-8 alongside numerical arrays.
- Keep native and comparison datasets in separate groups.
- Mark file finalization; incomplete files must be unmistakable.
- Publish an HDF5 schema version and migration tool for every incompatible change.

16.4 SWMR boundary

HDF5 SWMR permits a single process to write while multiple processes read, but official documentation requires compatible filesystem semantics and specific file-handling discipline [7]. CPTG-RX therefore treats SWMR as an optional same-filesystem evidence-streaming profile, not as the general live network protocol.

16.5 Evidence policy modes

Policy	Behavior
NONE	No frame evidence; session summary only. Prohibited for formal validation.
ERRORS_ONLY	Archive failed or warned frames.
SAMPLED	Archive deterministic sample plus all errors and event markers.
FULL	Archive every request and result.
HASH_ONLY	Archive canonical hashes and counters; raw states retained elsewhere.

17 Python/JAX Reference Implementation

17.1 Role

The Python implementation is the readable scientific authority and development environment. It is responsible for exact formulas, reference precision, vectorized evaluation, uncertainty propagation, automatic differentiation where authorized, schema generation, evidence writing, test-vector generation, and conformance testing.

17.2 Package layout

```
cptg_rx/
  authority/
    formulas/
    constants/
    manifests/
    registries/
  protocol/
    schemas/
    framing/
    control_plane/
    data_plane/
  reference_kernel/
    numpy_backend/
    jax_backend/
    exact_backend/
  runtime/
    server/
    client/
    shared_memory/
    zeromq/
  evidence/
    hdf5_writer/
```

```
    replay/
  conformance/
    vectors/
    validators/
    reports/
  benchmarks/
  adapters/
  cli/
  tests/
```

17.3 Direct API

```
from cptg_rx import Session, State

with Session.open(
    formula_id="CPTG.FORMULA.TBD",
    mode="EXACT",
    precision="FP64-STRICT",
    outputs=["native", "comparison", "jacobian"],
) as session:
    result = session.evaluate(State(...))
    result.require_valid()
```

17.4 JAX role

JAX may provide vectorization, differentiation, compilation, and portable export for formulas that map cleanly to its supported operations. JAX documents APIs for exporting and serializing staged computations and StableHLO artifacts [12,13]. For a compact closed-form formula, the directly audited C kernel remains the preferred strict runtime authority; JAX is valuable for reference calculations, derivatives, and larger differentiable workflows.

17.5 Reference implementation requirements

- Type-annotated public API and machine-readable schemas.
- No hidden global constants; all authority enters through a verified manifest.
- Deterministic canonical serialization and hash generation.
- Exact formula tests, numerical tests, property tests, invalid-input tests, and replay tests.
- Cross-backend comparison among exact, NumPy, JAX, and compiled implementations.
- Reproducible environment lock files and build metadata.
- Command-line tools for manifest inspection, server startup, replay, conformance, and benchmarking.

18 Deterministic Compiled Runtime Kernel

18.1 Purpose

After formula lock and reference validation, CPTG-RX shall provide a small deterministic runtime kernel with a stable C ABI. This kernel is the preferred integration point for hard-bounded same-process execution and for language bindings.

```
typedef struct {
    uint32_t abi_version;
    uint32_t formula_id;
    uint32_t flags;
    uint32_t input_count;
    const double *inputs;
    uint32_t output_mask;
    uint64_t sequence_number;
    uint64_t deadline_ns;
} cptg_rx_input_v1;

typedef struct {
    uint32_t abi_version;
    uint32_t status_code;
    uint32_t output_count;
    double *outputs;
    double *jacobian;
    double *hessian;
    uint64_t execution_time_ns;
```

```
    uint8_t formula_sha256[32];
} cptg_rx_output_v1;

int cptg_rx_evaluate_v1(
    const cptg_rx_input_v1 *input,
    cptg_rx_output_v1 *output
);
```

18.2 Runtime restrictions

- No allocation, filesystem access, network access, dynamic loading, or formula mutation during evaluation.
- No exceptions crossing the ABI boundary.
- All buffers are caller-provided and bounds-checked.
- Every return path sets a status code.
- Formula and constants are compiled or loaded once and hash-verified.
- No unbounded loops or recursion.
- Worst-case execution behavior is measured for each supported target.
- Compiler, flags, math library, and target architecture are part of the build manifest.

18.3 Language bindings

Python, C++, Rust, Fortran, Julia, MATLAB, and other bindings should wrap the same C ABI or the same generated FlatBuffers schema. A binding is conformant only if it passes the common test vectors.

19 External Facility and Model Adapters

19.1 Adapter responsibility

An adapter maps an institution-specific state into CPTG-RX and maps CPTG-RX outputs into the institution-specific transport, optimization, or control environment. The adapter is not allowed to rename or reinterpret native outputs without preserving the original field and formula authority.

19.2 Recommended adapter classes

Adapter	Use
Direct C ABI	Same-process deterministic integration.
Shared-memory ring buffer	Same-host process isolation with very low overhead.
ZeroMQ IPC/TCP	Flexible request/reply, publish/subscribe, and distributed facility integration. ZeroMQ supports atomic messages across in-process, inter-process, TCP, and other transports [4,5].
EPICS pvAccess	Integration with scientific control systems. EPICS defines pvAccess as a high-performance protocol for signal monitoring and structured scientific data services [14].
MARTe2	Integration into modular C++ real-time control applications. MARTe2 is documented as a C++ framework for real-time control-system development [15,16].
HTTP/OpenAPI gateway	Remote scalar requests, discovery, diagnostics, and institutional access.
MHD/transport connector	Maps source terms and derivatives into facility-specific solvers.

19.3 Adapter conformance boundary

- Unit conversion is explicit and tested.
- Species, reaction, grid, and output ordering are preserved.
- Stale sequence and validity masks are honored.
- Native and comparison values remain distinguishable.
- No out-of-domain result is converted into a valid facility value.

- Adapter latency and queue behavior are benchmarked separately from kernel latency.
- Facility actuator logic remains outside CPTG-RX.

20 Conformance and Certification

20.1 Conformance levels

Level	Name	Requirement
C0	Schema compatible	Parses and emits valid core messages.
C1	Scientific scalar	Matches locked scalar test vectors and failures.
C2	Vector/batch	Matches vectorized, profile, and partial-failure vectors.
C3	Derivative	Matches authorized Jacobian and Hessian vectors.
C4	Evidence/replay	Produces conformant HDF5 evidence and deterministic replay.
C5	Real-time runtime	Meets allocation, determinism, deadline, and benchmark requirements on a declared target.
C6	Facility adapter	Passes end-to-end sequence, unit, stale-data, and fault-injection tests with a named framework.

20.2 Conformance suite categories

- Formula and constants hash verification.
- Interior, boundary, and just-outside-domain reference evaluations.
- Exact zeros, sign changes, extreme authorized values, and near-cancellation cases.
- Cross-platform floating-point tolerance checks.
- Jacobian and Hessian validation.
- Malformed frames, unknown IDs, bad shapes, wrong units, NaNs, infinities, and oversize payloads.
- Stale sequences, replay attempts, missed deadlines, and overload behavior.
- Schema forward/backward compatibility cases.
- HDF5 structure, metadata, hash, and replay verification.
- Deterministic repeated evaluation and thread-safety tests.
- Latency, jitter, allocation, and throughput benchmarks.

20.3 Conformance report

Every conformance run produces a signed or hash-addressed report naming the protocol version, schema, formula, constants, software build, platform, compiler, precision, passed vectors, failed vectors, tolerances, and benchmark results. “CPTG-RX compatible” without a level and report is not a meaningful claim.

21 Benchmark and Performance Reporting

21.1 Required metrics

Metric	Definition
Cold-start time	Process start through SESSION_READY.
Warm scalar kernel latency	Kernel-only time after preload.
Warm end-to-end scalar latency	Adapter serialization, transport, queue, kernel, and response.
Vector throughput	Evaluations or state rows per second for declared shape.

Metric	Definition
p50/p95/p99/p99.9	Latency quantiles over a declared sample count.
Maximum observed	Largest observed latency; not a proof of worst-case bound.
Jitter	Distribution or range around expected cycle time.
Allocation per call	Dynamic bytes allocated in the hot path.
CPU/device utilization	Target-specific utilization and frequency policy.
Error vs reference	Maximum and RMS difference from exact/reference backend.
Overload behavior	Rejection, drop, queue, and recovery measurements.

21.2 Benchmark disclosure

A benchmark claim must identify the CPU/GPU, operating system, compiler or interpreter, runtime versions, thread count and affinity, power/frequency policy, formula and output mask, input shape, transport, serialization, evidence mode, warm-up, run count, and timing method. Kernel speed and end-to-end speed are reported separately.

21.3 Performance regression gate

Every protocol or implementation release runs the same benchmark suite. A regression beyond a declared threshold requires explicit approval and release notes. Correctness and fail-closed behavior take priority over speed.

22 Versioning, Governance, and Change Control

22.1 Independent version domains

Version	Controls
Protocol version	Core messages, session semantics, status model, and compatibility rules.
Schema version	Binary and JSON field definitions.
Formula version	Scientific expression and output definitions.
Constants version	Numerical/exact constant authority.
Registry version	Species, reaction, observable, and comparison-coordinate IDs.
Software version	Reference server, client, runtime kernel, or adapter implementation.
Evidence schema version	HDF5 group and dataset organization.

22.2 Compatibility rules

- Major protocol changes may be incompatible and require explicit negotiation.
- Minor protocol changes may add optional features without changing existing meanings.
- Schema fields are appended or deprecated according to published compatibility rules; core field IDs are never reused.
- A scientific formula change always receives a new formula version and hash, even if the API shape is unchanged.
- A constants change receives a new constants version and hash.
- Deprecated formulas remain identifiable and replayable but may be prohibited for new live sessions.
- Withdrawn formulas remain in the historical registry with a withdrawal reason.

22.3 Change proposal process

17. Submit a protocol change proposal with motivation, compatibility impact, schemas, tests, and security/performance analysis.
18. Review scientific effects separately from transport or implementation effects.

19. Publish an experimental extension namespace and conformance vectors.
20. Run reference, cross-language, evidence, and fault-injection tests.
21. Approve, revise, or reject through documented governance.
22. Assign final versions, hashes, migration guidance, and deprecation policy.

23 Implementation Roadmap Under Locked Authority

Phase	Deliverable
Phase 0 - Scientific authority (complete)	Universal light-element theory, state roles, transport-polarization structure, constants authority, domains, outputs, and validation boundary are treated as closed. Later phases implement and qualify the exchange protocol without altering the theory.
Phase 1 - Normative specification	Freeze CPTG-RX/1.0 core messages, registries, status codes, units, canonicalization, and HDF5 schema.
Phase 2 - Python reference	Implement exact/NumPy/JAX backends, manifests, JSON/OpenAPI control plane, FlatBuffers schemas, HDF5 writer, replay, and test vectors.
Phase 3 - C runtime kernel	Generate or hand-audit the deterministic C ABI kernel, cross-check every vector, and measure target-specific timing.
Phase 4 - Transport profiles	Implement direct, shared-memory, ZeroMQ, and HTTP profiles with identical payload semantics.
Phase 5 - Conformance kit	Release independent clients, malformed-input corpus, cross-platform vectors, evidence validators, and benchmark harness.
Phase 6 - Institutional pilot	Integrate with one or more external models in offline replay and compare against archived datasets.
Phase 7 - Live shadow service	Ingest live telemetry and emit results without operator recommendation or actuator influence.
Phase 8 - Advisory/supervisory research	Permit bounded recommendations or slow supervisory use only after facility-specific validation and independent review.

23.1 Locked-authority handoff checklist

- Final formula identifiers and controlled authority payloads.
- Constants and exact values.
- Input and output dictionary with units and dimensions.
- Authorized scalar and multidimensional domains.
- Derivative definitions and availability.
- Numerical precision and tolerance requirements.
- Reference and held-out vectors.
- Known limitations and prohibited extrapolations.
- Conservation and invariant checks.
- Formula, constants, and vectors hashes.

24 Deployment Maturity and Safety Boundary

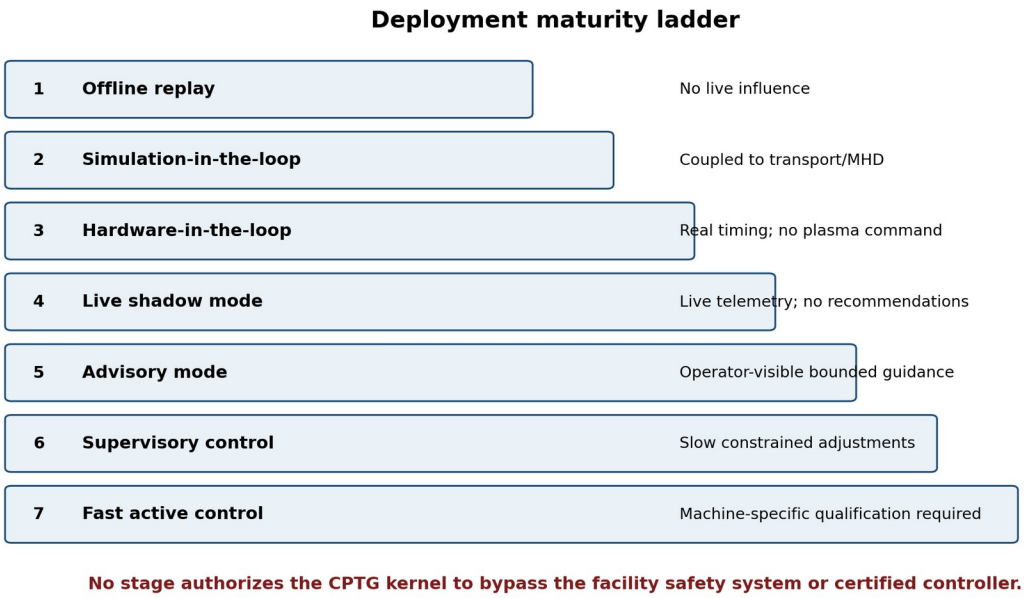

Figure 4. CPTG-RX progresses from offline evidence to possible live use only through explicit maturity stages.

24.1 Qualification principle

Protocol conformance is not facility safety qualification. A conformant CPTG-RX implementation proves that it exchanges and evaluates the locked scientific contract correctly. A facility must separately validate control stability, timing, fault behavior, cybersecurity, operational procedures, and regulatory requirements.

24.2 Fallback behavior

If CPTG-RX is unavailable, late, invalid, out of domain, or hash-inconsistent, the facility system must fall back to its preexisting safe behavior. CPTG-RX must never be the sole protection path.

24.3 Live-use constraints

- Every live output includes validity and age.
- No output is applied after its deadline.
- No warning or error is hidden by the adapter.
- Formula and build hashes are monitored during operation.
- Control authority is bounded by the facility, not by CPTG-RX.
- All live deployment stages have an immediate disable path.

25 End-to-End Exchange Example

25.1 Control-plane session request

```
POST /cptg-rx/v1/sessions
{
  "protocol": "CPTG-RX/1.0",
  "formula_id": "CPTG.DYNAMIC_SOURCE_CURRENT.LIGHT4",
  "formula_sha256": "<locked digest>",
  "mode": "REALTIME",
  "precision": "FP64-STRICT",
  "transport": "ZEROMQ_IPC",
  "schema_id": 1001,
  "requested_outputs": [
    "native.reactivity",
    "native.reaction_rate",
    "native.species_source",
    "derivatives.jacobian",
    "comparison.reference_rate"
  ],
```

```
  "maximum_profile_points": 256,
  "deadline_policy": "REJECT_LATE",
  "evidence_policy": "SAMPLED"
}
```

25.2 Session-ready response

```
{
  "session_id": 843128613,
  "status": "READY",
  "formula_status": "LOCKED",
  "formula_sha256": "<locked digest>",
  "constants_sha256": "<locked digest>",
  "schema_sha256": "<schema digest>",
  "preload_complete": true,
  "self_test": "PASS",
  "maximum_frame_bytes": 262144,
  "endpoint": "ipc:///run/cptg_rx/session_843128613"
}
```

25.3 Repeated data-plane exchange

The client sends StateFrame sequence 1 with canonical units, shapes, requested outputs, and deadline. The server verifies the session formula hash, evaluates only the requested outputs, and returns ResultFrame sequence 1. The client rejects any result with a different sequence, invalid domain mask, late completion, or non-OK status.

25.4 Evidence record

Under SAMPLED evidence policy, selected valid frames plus every warning, error, and event marker are written to the session .cptg.h5 file. Session close returns the finalized evidence hash and timing summary.

25.5 Example result semantics

```
ResultFrame {
  sequence_number: 1,
  status_code: OK,
  native_outputs: {...},
  comparison_outputs: {
    reference_model_id: "INSTITUTE.MODEL.VERSION",
    transform_definition_id: "CPTG.TRANSFORM.LIGHT4.1",
    correction_factor: ...,
    residual: ...
  },
  jacobian: {...},
  uncertainty: {
    numerical: ...,
    input: ...,
    observational: ...,
    domain: ...
  },
  domain_validity_mask: ALL_VALID,
  formula_sha256: <locked digest>,
  execution_time_ns: ...
}
```

26 CPTG-RX/1.0 Release Acceptance Criteria

Gate	Acceptance requirement
Scientific authority	At least one formula package is LOCKED with exact expression, constants, domain, output definitions, validation record, and conformance vectors.
Specification	Core messages, profiles, status codes, units, canonicalization, versioning, and compatibility rules are frozen.
Schemas	OpenAPI, JSON, FlatBuffers, C ABI, registries, and HDF5 schema are published and hash-addressed.

Gate	Acceptance requirement
Reference implementation	Python implementation passes all exact, numerical, invalid-input, evidence, and replay tests.
Compiled runtime	C ABI runtime matches reference vectors within the declared tolerance.
Conformance kit	At least two independent implementations or language clients pass the required conformance level.
Evidence	A complete .cptg.h5 example independently replays to the expected results.
Security	Threat model, authentication, authorization, TLS profile, replay protection, and dependency review are complete.
Performance	Reproducible kernel and end-to-end benchmark reports are published.
Documentation	Integrator guide, formula guide, adapter guide, error catalogue, and migration policy are complete.
Governance	Change-control, deprecation, withdrawal, registry, and release-signing procedures are assigned.
Boundary	Documentation states that CPTG-RX is not a certified controller and has no direct actuator authority.

RELEASE RULE CPTG-RX/1.0 shall not be declared production-ready merely because the formula evaluates quickly. Scientific authority, failure behavior, unit discipline, reproducibility, security, and interoperability are co-equal release gates.

Appendix A Core Field Dictionary

The final specification shall publish a machine-readable registry. The table below defines the minimum classes that must exist before implementation.

Registry	Required fields
Species	numeric ID, canonical name, isotope notation, charge, mass authority, status, aliases
Reaction	numeric ID, ordered reactants, ordered products, direction, symmetry factor, Q-value authority, status
Observable	numeric ID, name, tensor rank, shape rules, unit, sign convention, native/comparison class
Coordinate	numeric ID, model or database owner, definition, unit, transform ID, uncertainty convention
Distribution	numeric ID, physical meaning, parameter order, normalization, valid domain
Grid	numeric ID, coordinate system, ordering, boundaries, interpolation policy
Status/warning	numeric code, name, severity, retryability, operator meaning

Appendix B Formula Manifest Example

```
{
  "formula_id": "CPTG.DYNAMIC_SOURCE_CURRENT.LIGHT4",
  "formula_version": "1.0.0",
```

```
  "formula_status": "LOCKED",
  "authority_payload": "<controlled immutable payload>",
  "disclosure_policy": "CONTROLLED",
  "formula_sha256": "<digest>",
  "constants": {
    "manifest_uri": "authority/constants-1.0.0.json",
    "sha256": "<digest>"
  },
  "authorized_domain": {
    "inputs": [...],
    "reactions": [...],
    "grids": [...]
  },
  "outputs": [...],
  "derivatives": {
    "jacobian": true,
    "hessian": true
  },
  "precision_contract": {
    "reference": "EXTENDED",
    "runtime": "FP64-STRICT",
    "absolute_tolerance": "<locked>",
    "relative_tolerance": "<locked>"
  },
  "validation": {
    "record_id": "<locked evidence identifier>",
    "held_out_status": "PASS"
  },
  "test_vectors_sha256": "<digest>"
}
```

Appendix C FlatBuffers Schema Skeleton

```
namespace CPTG.RX.V1;

enum MessageType:ushort {
  HELLO = 1,
  CAPABILITIES = 2,
  FORMULA_MANIFEST = 3,
  SESSION_OPEN = 4,
  SESSION_READY = 5,
  STATE_FRAME = 10,
  RESULT_FRAME = 11,
  ERROR_FRAME = 12,
  HEARTBEAT = 13,
  SESSION_CLOSE = 20,
  SESSION_SUMMARY = 21
}

table TensorF64 {
  shape:[uint];
  values:[double];
  unit_id:uint;
  axis_ids:[uint];
}

table StateFrame {
  sequence_number:ulong;
  measurement_timestamp_ns:ulong;
  validity_interval_ns:ulong;
  reaction_channel_ids:[uint];
  species_ids:[uint];
  fields:[TensorF64];
  requested_output_mask:ulong;
}

table ResultFrame {
  sequence_number:ulong;
  status_code:uint;
  execution_time_ns:ulong;
  native_outputs:[TensorF64];
  comparison_outputs:[TensorF64];
  jacobian:[TensorF64];
  hessian:[TensorF64];
  domain_validity_mask:[ubyte];
  formula_sha256:[ubyte];
  result_hash:[ubyte];
}
```

```
root_type ResultFrame;
```

Appendix D HDF5 Replay Requirements

- A replay tool verifies authority hashes before reading scientific outputs.
- Replay reconstructs every StateFrame in original sequence order.
- Replay uses the recorded formula and precision profile.
- Each calculated result is compared with archived expected outputs and tolerances.
- Row-level validity, warning, and error codes must match.
- Timing is reported but is not required to match across hardware.
- A replay report records exact, tolerance, hash, and structural pass/fail separately.
- Any migrated evidence file retains the original schema and migration provenance.

Appendix E Minimum Fault-Injection Corpus

- Wrong magic bytes
- Unsupported protocol major
- Unknown schema
- Declared size smaller/larger than payload
- Wrong session ID
- Duplicate sequence
- Sequence regression
- Expired state
- Deadline already passed
- Formula hash mismatch
- Constants hash mismatch
- Unknown species ID
- Unknown reaction ID
- Missing units
- Unit-dimension mismatch
- Array shape mismatch
- Empty required array
- NaN input
- Infinity input
- Out-of-domain low boundary
- Out-of-domain high boundary
- Numerical overflow
- Invariant violation
- Evidence writer unavailable
- Transport disconnect
- Overload queue full
- Malformed FlatBuffer
- Unauthorized formula
- Withdrawn formula
- Result arrives after client cycle advanced

Appendix F Initial Repository and Release Artifacts

```
cptg-rx-spec/
  specification.pdf
  specification.md
  openapi.yaml
  schemas/
    cptg_rx_v1.fbs
    json/
    hdf5_schema.yaml
  registries/
```

```
authority/
reference-python/
runtime-c/
clients/
  python/
  cpp/
  rust/
adapters/
  epics/
  marte2/
  zeromq/
  shared_memory/
conformance/
  vectors/
  malformed/
  replay/
benchmarks/
examples/
security/
governance/
LICENSE
CHANGELOG.md
```

Appendix G References

[1] OpenAPI Initiative. OpenAPI Specification v3.2.0. https://spec.openapis.org/oas/v3.2.0.html Language-agnostic HTTP API description.

[2] FlatBuffers Project. FlatBuffers: Evolution. https://flatbuffers.dev/evolution/ Schema evolution and compatibility rules.

[3] FlatBuffers Project. FlatBuffers White Paper and Internals. https://flatbuffers.dev/white_paper/ In-place structured binary format and platform representation.

[4] ZeroMQ Project. ZeroMQ. https://zeromq.org/ Atomic messaging across multiple transports and patterns.

[5] ZeroMQ Project. ZeroMQ Guide: Sockets and Patterns. https://zguide.zeromq.org/docs/chapter2/ Messaging and asynchronous I/O patterns.

[6] The HDF Group. The HDF5 Library and File Format. https://www.hdfgroup.org/solutions/hdf5/ Scientific data management and storage.

[7] The HDF Group. Introduction to Single-Writer/Multiple-Reader (SWMR). https://support.hdfgroup.org/documentation/hdf5/latest/_s_w_m_r_t_n.html SWMR behavior and filesystem requirements.

[8] Bureau International des Poids et Mesures. The International System of Units (SI Brochure), 9th edition. https://www.bipm.org/en/publications/si-brochure Canonical units and measurement traceability.

[9] National Institute of Standards and Technology. FIPS 180-4: Secure Hash Standard. https://csrc.nist.gov/pubs/fips/180-4/upd1/final SHA-2 hash algorithms including SHA-256.

[10] RFC Editor / IETF. RFC 3339: Date and Time on the Internet: Timestamps. https://www.rfc-editor.org/info/rfc3339/ Internet timestamp profile.

[11] RFC Editor / IETF. RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3. https://www.rfc-editor.org/info/rfc8446/ Remote transport security.

[12] JAX Project. Exporting and Serializing Staged-Out Computations. https://docs.jax.dev/en/latest/export/export.html Portable export and StableHLO considerations.

[13] JAX Project. Ahead-of-Time Lowering and Compilation. https://docs.jax.dev/en/latest/aot.html Tracing, lowering, compilation, and execution workflow.

[14] EPICS. pvAccess Protocol Specification. https://docs.epics-controls.org/en/latest/pv-access/protocol.html High-performance scientific signal and structured-data protocol.

[15] Fusion for Energy. MARTe2 Documentation. https://vcis.f4e.europa.eu/marte2-docs/master/html/ Real-time framework documentation.

[16] MARTe2 Project. MARTe2 Source Repository. https://github.com/aneto0/MARTe2 C++ modular real-time control framework source and overview.

Appendix H Locked Universal Light-Element Theory Profile

This profile defines the public protocol-facing description of the final CPTG nuclear-reaction extension. It communicates the theory's state roles, transport structure, output classes, validity boundary, and evidence requirements without reproducing the protected equations or normalization details.

H.1 Scientific scope

The authorized light-element system is organized as one ordered geometric sequence: hydrogen is the vertex state, deuterium is the bridge state, the mass-three sector is the closure state, and helium-4 is the saturation state. During active reaction evolution, the vertex includes free neutrons and free protons, while the closure sector retains both tritium and helium-3 as internal orientations of the same mass-three role.

H.2 Transport-polarization contract

The public contract exposes three ordered transport directions - vertex to bridge, bridge to closure, and closure to saturation - together with one independent charge-constrained polarization direction. Reaction channels enter the protocol through stable source identifiers. The runtime returns the network response in the same transport-polarization coordinate system, plus separately named abundance, rate, sensitivity, and comparison outputs when authorized by the manifest.

H.3 Authority and disclosure boundary

The theory is closed, but the public exchange specification does not disclose its equations. Every result is bound to a controlled authority package containing the immutable scientific payload, constants authority, authorized domain, precision contract, output definitions, validation record, and conformance hashes. Clients verify the authority package by identifier and digest; they do not reconstruct or modify the theory.

H.4 Required public registries

Registry	Required public meaning
State roles	Vertex, bridge, closure, saturation; stable IDs and human-readable labels.
Transport directions	Three ordered edge identifiers with sign and orientation conventions.
Polarization	One independent internal orientation identifier and charge-consistency status.
Reaction channels	Stable reaction IDs, ordered reactants/products, source class, and release status.
Outputs	Native, comparison, derivative, uncertainty, timing, validity, and evidence identifiers.

NON-DISCLOSURE RULE Protocol documentation may describe the architecture and output semantics, but shall not publish the protected formula payload without a separately authorized scientific release.

Appendix I Implementation and Qualification Status

Revision r02 moves CPTG-RX from pre-formula planning into locked-theory implementation authority. The theory is not reopened by software testing. Implementation work verifies that independent runtimes reproduce the controlled authority, remain numerically stable, and fail closed under malformed or adversarial conditions.

I.1 Current authority status

Scientific theory	Closed universal light-element architecture.
Public formula disclosure	Not authorized by this framework.
Protocol specification	Implementation authority; core interfaces remain version controlled.
Reference runtime	Python/JAX authority implementation and deterministic compiled kernel profile.
Evidence profile	HDF5 replay, immutable hashes, manifests, and exact environment provenance.
Facility control authority	Explicitly outside CPTG-RX; remains with the institution.

I.2 Qualification campaigns

The qualification program contains two complementary tracks. The native reaction-network campaign measures independent discovery and held-out reaction channels across the full light-element state and transport-polarization response. The numerical-rigidity campaign applies large randomized ensembles, direct matrix propagation, malformed inputs, underflow conditions, simulated corruption, invariant monitoring, deterministic replay, and fail-closed shutdown tests. Neither campaign is permitted to alter the locked theory after held-out exposure.

I.3 Release boundary

A protocol build may be released only when its authority hashes, schemas, registries, precision profile, conformance vectors, replay files, malformed-input corpus, security controls, benchmark report, and migration rules are complete. A fast kernel alone is insufficient. Facility deployment additionally requires independent timing, stability, cybersecurity, operational, and regulatory qualification.

FINAL THEORY STATUS The universal CPTG light-element reaction architecture is treated as closed. Remaining activity concerns implementation, conformance, replication, publication, and deployment qualification. Formula changes require a separately governed scientific revision and cannot be introduced through adapters, transports, or runtime optimization.